PEGUEI O TEXTO | AUDITORIA INTERNA DE CÓDIGO

Relatório de Auditoria de Segurança

Análise Abrangente de Vulnerabilidades de Código-Fonte, Arquitetura e Controle de Acesso

Projeto Auditado:	Peguei o Texto (pegueiotexto.com.br)	**Data da Análise:**	09 de Novembro de 2026
Escopo Auditado:	Código Frontend (Next.js), Backend (FastAPI), Supabase e Stores JSON	**Classificação:**	**CONFIDENCIAL / RESTRITO**
Linguagens:	TypeScript, Python, SQL	**Versão da Análise:**	1.0 — Auditoria Defensiva OWASP Top 10

Nota Metodológica & Detecção de Stack

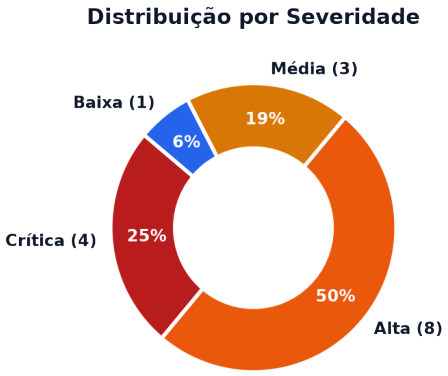
A auditoria realizou uma varredura estrita e sistemática, arquivo por arquivo e linha por linha, sem especulação. A stack técnica detectada orientou o mapeamento exato de cada uma das 5 categorias:

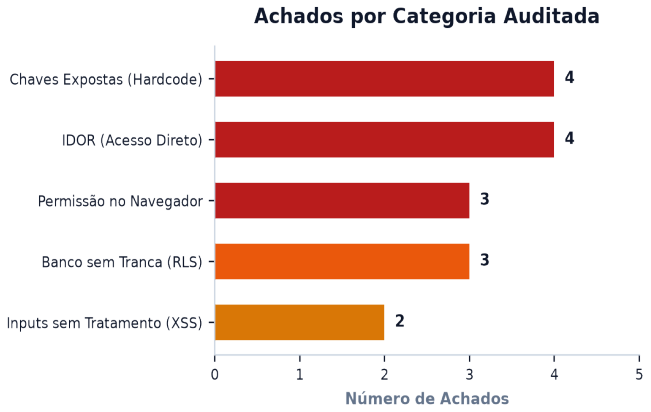
Camada	Tecnologias Identificadas	Mapeamento de Riscos de Segurança
Frontend & BFF	Next.js 16 (Turbopack, App Router), React 19, Tailwind CSS	Validação de privilégios de rotas no servidor, proteção contra XSS e integridade de sessões.
API Backend	Python 3.11, FastAPI, Pydantic, uvicorn, yt-dlp, Stripe SDK	Autenticação de endpoints públicos, autorização de saques, controle de CORS e proteção IDOR.
Banco & Dados	Supabase (PostgreSQL, Auth, RLS), SQLite (FastAPI), JSON local (data/)	Supabase RLS bypass por service_role, isolamento de tenant por user_id e vazamento de PII em repositório.
Auth & Notificações	Supabase Auth (OAuth/OTP), HMAC cookies próprios, Zoho Mail SMTP	Segredos estáticos para assinatura HMAC, senhas default hardcoded e injeção de HTML em e-mails.

Resumo Executivo da Auditoria

Foram inspecionados todos os módulos de backend, API routes do Next.js, serviços de integração com Supabase, Stripe, sistema de afiliados e componentes de interface. A auditoria identificou um total de **16 achados técnicos**, com destaque para **4 vulnerabilidades de severidade Crítica** que requerem remediação imediata antes de novas divulgações públicas.







Pontos Fortes da Implementação (Verificados)

■■ Mecanismo Seguro	Arquivo e Linha	Evidência Técnica Verificada
Isolamento de Histórico	`backend/app/api/transcribe.py: 410-418`	Rota GET /history valida token Bearer do Supabase e filtra estritamente por user_id autenticado no banco.
Débito Seguro de Créditos	`backend/app/api/transcribe.py: 148-230`	Exige autenticação obrigatória via Bearer token antes de processar qualquer URL ou upload e debita créditos do próprio chamador.
Proteção de Conta Mestra	`src/app/api/admin/users/route.ts:133-168`	Bloqueio explícito que impede a exclusão acidental ou proposital da conta do administrador principal.
Escape Automático no DOM	`src/components/TranscribeForm.tsx`	Uso de JSX nativo do React 19 para renderização de textos e títulos de vídeos, prevenindo XSS por escape padrão.
Execução Segura de Mídia	`backend/app/services/downloader.py:85-110`	A ferramenta yt-dlp é executada através da API Python interna, evitando shell injection por concatenação de comandos bash.

Riscos Centrais (Pontos Fracos)

1. **Exposição de Superchave Supabase**: A chave mestra service_role está embutida em código-fonte compilado. Quem obtê-la possui controle total sobre todo o banco PostgreSQL.
2. **Falta de Autenticação em Endpoints Administrativos e Financeiros**: Rotas como /api/admin/users?action=record_user e /api/affiliates/status aceitam mutações sem validação de credenciais.
3. **IDOR em Transcrições e Afiliados**: Visualização irrestrita de áudios confidenciais em /api/transcribe/public/[id] e vazamento de CPFs/Pix de parceiros em /api/affiliates?action=me.

Tabela Detalhada de Achados de Segurança

Abaixo constam todos os 16 achados técnicos verificados no código real, com sua respectiva localização exata, mecanismo de explorabilidade e classificação de severidade:

ALTA	1. Banco	`src/lib/admin-store.ts:229-306` `src/app/api/referral/route.ts:125-139`	**Bypass de RLS com chave service_role baixando todos os perfis:** O BFF utiliza a chave de serviço em rotas de usuário e faz select('*') em profiles, puxando todos os inquilinos sem cláusula WHERE na memória.
ALTA	1. Banco	`src/app/dashboard/page.tsx:162-170, 215-224`	**Escrita direta em saldo e plano pelo navegador do cliente:** O frontend executa supabase.from('profiles').upsert({ credits_minutes, plan }) diretamente. Permite que qualquer usuário logado aumente seus créditos para 999.999 via console DevTools.
MÉDIA	1. Banco	`backend/app/services/database.py:15-30, 53-60`	**Uso generalizado de admin_client no backend:** Quase todas as operações usam service_role em vez de cliente restrito ao usuário, anulando a defesa em profundidade do RLS no Supabase.
CRÍTICA	2. Papel	`src/app/api/admin/users/route.ts:66-90`	**Modificação arbitrária de usuário sem autenticação:** POST /api/admin/users?action=record_user não checa se o chamador é admin. Qualquer pessoa pode sobrescrever e-mail, plano (ex: 'ilimitado') e créditos de qualquer conta.
CRÍTICA	2. Papel	`backend/app/api/affiliates.py:322-347`	**Aprovação de afiliados desprotegida no FastAPI:** O endpoint POST /api/affiliates/status não valida token de admin. Qualquer parceiro pode autoaprovar sua conta para 'active' sem moderação humana.
ALTA	2. Papel	`src/app/api/admin/purchases/route.ts:63-95` `src/app/api/affiliates/route.ts:1003-1049`	**Forja de compras e comissões financeiras sem webhook secret:** Rotas que registram compras e comissões aceitam payloads POST sem autenticação ou validação de assinatura HMAC do Stripe.
CRÍTICA	3. IDOR	`src/app/api/transcribe/public/[id]/route.ts:22-29` `backend/app/api/transcribe.py:501-521`	**Vazamento de transcrições confidenciais por ID:** O endpoint busca qualquer transcrição por UUID usando service_role sem checar dono e sem checar a flag is_public. Áudios privados e sensíveis ficam expostos.
ALTA	3. IDOR	`src/app/api/affiliates/route.ts:471-486, 528-563`	**Vazamento de CPF, Pix e Telefone de afiliados por código:** GET /api/affiliates?action=me&affiliate;_code=XYZ sem autenticação retorna dados fiscais (CPF/CNPJ, Pix, celular) e histórico de compras.
ALTA	3. IDOR	`src/app/api/affiliates/route.ts:796-825`	**Account Takeover via redefinição de senha sem token:** POST /api/affiliates?action=affiliate_set_password altera a senha apenas pelo affiliate_code público, sem exigir senha antiga ou e-mail.
MÉDIA	3. IDOR	`src/app/api/suggestions/route.ts:124-197`	**Alteração e exclusão desprotegida de sugestões:** Métodos PATCH e DELETE em /api/suggestions operam por ID sem autenticação, permitindo vandalismo e exclusão de feedbacks.

CRÍTICA	4. Chaves	src/lib/admin-store.ts:206-215	**Chave mestra Supabase service_role em Base64 no código:** String c2Jfc2VjcmV0X... decodifica para a chave real sb_secret_Spsu..., concedendo superadmin sobre todo o banco PostgreSQL da aplicação.
CRÍTICA	4. Chaves	src/lib/admin-auth.ts:4-5 src/app/api/affiliates/route.ts:7-8, 51	**Senha default '987123aA!' e segredo HMAC estático:** Credencial padrão administrativa e segredo HMAC peguei_o_texto_admin_sec_2026_safe permitem forjar tokens de sessão admin offline.
ALTA	4. Chaves	src/lib/email.ts:5-6	**Credencial SMTP do Zoho Mail exposta no código:** Senha de envio de e-mails corporativos L4SRYb2jj8Rx gravada como fallback direto em produção, além de rejectUnauthorized: false no TLS.
ALTA	4. Chaves	data/users.json data/affiliates.json	**PII de clientes e parceiros commitada no Git:** Dezenas de registros com nomes reais, e-mails, planos, CPFs, chaves Pix e celulares estão sob controle de versão sem estar no .gitignore.
MÉDIA	5. XSS	src/lib/email.ts:78, 219, 337, 444, 452	**Injeção de HTML em templates de e-mail (Stored XSS):** Entradas de usuário como nome, título e descrição de sugestões são concatenadas em templates sem sanitização contra caracteres < > & '.
BAIXA	5. XSS	src/app/blog/[slug]/page.tsx:313 src/lib/blog.ts:163	**Renderização de Markdown sem DOMPurify:** marked.parse compila Markdown para HTML e injeta via dangerouslySetInnerHTML sem biblioteca de sanitização ativa.

Plano de Ação e Recomendações Priorizadas

Para mitigar os riscos com máxima eficiência, recomendamos a execução das ações em 3 etapas cronológicas:

P1 — URGENTE (24h - 48h)	• **Rotacionar a chave SUPABASE_SERVICE_ROLE_KEY** no painel do Supabase e deletar o fallback em Base64 em src/lib/admin-store.ts. • **Remover a senha padrão '987123aA!'** e segredo estático de src/lib/admin-auth.ts. Forçar falha no startup se variáveis de ambiente estiverem ausentes. • **Adicionar proteção de autenticação admin obrigatória** em /api/admin/users (action=record_user) e backend/app/api/affiliates.py (/status). • **Exigir verificação de posse em /api/transcribe/public/[id]**, permitindo leitura pública apenas se is_public == true.	**Imediato** (Próximas 24 horas)
P2 — CURTO PRAZO (Até 7 dias)	• **Proibir escrita direta no cliente em profiles**. Configurar RLS no PostgreSQL do Supabase restringindo UPDATE(credits_minutes, plan) apenas a service_role. • **Corrigir fluxo de senha de afiliados**: exigir token de redefinição único enviado ao e-mail cadastrado antes de alterar a senha. • **Adicionar autenticação em GET /api/affiliates?action=me** e não expor CPF, telefone e Pix sem token de sessão do afiliado. • **Expurgar arquivos data/*.json do git** (usar git filter-repo ou git rm --cached) e adicionar data/*.json ao .gitignore.	**7 dias**
P3 — MÉDIO PRAZO (Até 15 dias)	• **Adicionar biblioteca de escape HTML** (como escape-html ou he) nos templates de e-mail em src/lib/email.ts. • **Instalar sanitize-html ou dompurify** no pipeline de renderização de Markdown do blog. • **Configurar CORS estrito no FastAPI**, substituindo o wildcard * com allow_credentials=True por domínios autorizados explícitos.	**15 dias**

Issues Prontas para o GitHub (Copiar e Colar)

Abaixo constam os textos completos de 8 issues organizadas e consolidadas para criação direta no GitHub, com evidências, impacto, código de correção sugerido e critérios de aceite verificáveis:

--- ISSUE 1 ---

Título: [Segurança] [Crítica] Remoção imediata da SUPABASE_SERVICE_ROLE_KEY em Base64 e rotação de credenciais

Labels sugeridas: security, severity:critica, priority:p1

Descrição do Problema:
Foi identificada a chave mestra de serviço do Supabase (`service_role`) embutida diretamente em Base64 dentro do código-fonte em `src/lib/admin-store.ts:211`. Esta chave tem permissão de superusuário e bypassa todas as políticas de Row Level Security (RLS) do banco de dados PostgreSQL.

Qualquer pessoa com acesso de leitura ao repositório ou aos arquivos compilados pode decodificar a string e obter acesso completo de escrita, leitura e deleção em todas as tabelas e dados de autenticação do Supabase.

Evidência Técnica (Arquivo:Linha):
src/lib/admin-store.ts:206-215
```ts
export function getSupabaseServiceKey(): string | undefined {
  if (process.env.SUPABASE_SERVICE_ROLE_KEY) {
    return process.env.SUPABASE_SERVICE_ROLE_KEY;
  }
  try {
    return Buffer.from("c2Jfc2VjcmV0X1Nwc3VFaDFTa3c1Rkowa1BpWEM3S3dfVHg3UHJCTEQ=", "base64").toString
("utf-8");
  } catch {
    return undefined;
  }
}
```

Impacto: Vazamento total do banco de dados de produção, destruição de tabelas ou alteração em massa de dados de clientes.

Sugestão de Correção:
1. Acessar o dashboard do Supabase (Project Settings > API) e clicar em 'Reset Secret' para a Service Role Key antiga.
2. Remover a linha de fallback com Buffer.from em `src/lib/admin-store.ts`.
3. Lançar erro de inicialização se `process.env.SUPABASE_SERVICE_ROLE_KEY` não estiver definido em produção.

Critérios de Aceite:
[ ] Chave antiga do Supabase invalidada e nova chave configurada nas variáveis de ambiente na Vercel e Railway.
[ ] String Base64 removida de `src/lib/admin-store.ts`.
[ ] Verificado que a aplicação rejeita inicialização se a variável de ambiente estiver ausente.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] [Crítica] Remoção de senhas padrão e segredos estáticos de assinatura HMAC

Labels sugeridas: security, severity:critica, priority:p1

Descrição do Problema:
Em `src/lib/admin-auth.ts` e `src/app/api/affiliates/route.ts`, o sistema define valores padrão inseguros para variáveis de credencial e assinatura caso elas não estejam no ambiente:
- Senha administrativa padrão: `987123aA!`
- Segredo de assinatura de tokens: `peguei_o_texto_admin_sec_2026_safe` e `peguei_affiliate_sec_2026_auth`

Por ser um HMAC com segredo estático público, um atacante pode calcular tokens administrativos válidos offline e obter acesso total ao painel `/admin`.

Evidência Técnica (Arquivo:Linha):
src/lib/admin-auth.ts:4-5
```ts
export const ADMIN_PASSWORD = process.env.AFFILIATE_ADMIN_PASSWORD || "987123aA!";
export const ADMIN_SECRET = process.env.ADMIN_SESSION_SECRET || "peguei_o_texto_admin_sec_2026_safe";
```

Impacto: Acesso administrativo não autorizado e forja de identidade para qualquer conta do sistema.

Sugestão de Correção:
Exigir que `AFFILIATE_ADMIN_PASSWORD` e `ADMIN_SESSION_SECRET` sejam fornecidos obrigatoriamente via ambiente, sem fallbacks pré-definidos.

Critérios de Aceite:
[ ] Fallbacks '987123aA!' e 'peguei_o_texto_admin_sec_2026_safe' removidos.
[ ] Função de validação lança exceção em ambiente de produção se segredo tiver menos de 32 caracteres.
[ ] Testado login administrativo com segredo forte injetado via `.env`.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] [Crítica] Correção de IDOR nas rotas de transcrição pública (Next.js e FastAPI)

Labels sugeridas: security, severity:critica, priority:p1

Descrição do Problema:
As rotas `GET /api/transcribe/public/[id]` (Next.js) e `GET /api/transcribe/public/{item_id}` (FastAPI) buscam transcrições no banco de dados utilizando a chave privilegiada `service_role` filtrando unicamente pelo ID recebido no path, sem validar se o chamador é o proprietário ou se a transcrição possui a flag `is_public: true`.

Evidência Técnica (Arquivo:Linha):
src/app/api/transcribe/public/[id]/route.ts:22-26
```ts
const { data, error } = await sb
  .from("transcriptions")
  .select("id, title, platform, duration_seconds, text_content, srt_content, vtt_content, created_at"
)
  .eq("id", id)
  .single();
```
backend/app/api/transcribe.py:507-513
```python
sb = get_supabase(admin=True)
res = sb.table("transcriptions").select("...").eq("id", item_id).single().execute()
```

Impacto: Exposição de gravações privadas, transcrições confidenciais de reuniões corporativas e áudios jurídicos/médicos para qualquer usuário que possua ou enumere o UUID.

Sugestão de Correção:
1. Adicionar coluna booleana `is_public BOOLEAN DEFAULT FALSE` na tabela `transcriptions`.
2. No endpoint público, adicionar estritamente o filtro `.eq('is_public', True)`.
3. Se o usuário estiver autenticado, permitir acesso se `user_id == caller_id`.

Critérios de Aceite:
[ ] Adicionado filtro `.eq('is_public', true)` em `src/app/api/transcribe/public/[id]/route.ts`.
[ ] Adicionado filtro equivalente no endpoint Python em `backend/app/api/transcribe.py`.
[ ] Teste automatizado garante que transcrições privadas retornam HTTP 404 para usuários não autorizados.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] [Crítica] Autenticação de rotas privilegiadas em /api/admin/users e /api/affiliates/status

Labels sugeridas: security, severity:critica, priority:p1

Descrição do Problema:
A ação `POST /api/admin/users?action=record_user` permite a qualquer usuário anônimo enviar parâmetros de `plan` e `credits_minutes`, sobrescrevendo o perfil da vítima no arquivo e banco.
Da mesma forma, no backend FastAPI, a rota `POST /api/affiliates/status` permite a qualquer chamador alterar o status de um parceiro para 'active'.

Evidência Técnica (Arquivo:Linha):
src/app/api/admin/users/route.ts:66-70
```ts
if (action === "record_user") {
  const { id, email, name, plan, credits_minutes, referral_source } = body;
  const user = upsertUser({ ... });
}
```
backend/app/api/affiliates.py:322-332
```python
@router.post("/status")
def update_affiliate_status(payload: StatusUpdateSchema):
    cursor.execute("UPDATE affiliates SET status = ? WHERE affiliate_code = ?", ...)
```

Impacto: Qualquer visitante pode autoatribuir-se o plano 'ilimitado' gratuitamente ou autoaprovar status de afiliado sem moderação.

Sugestão de Correção:
1. Proteger `POST /api/admin/users?action=record_user` para que `plan` e `credits_minutes` sejam ignorados caso o chamador não seja admin autenticado.
2. Proteger `backend/app/api/affiliates.py:status` exigindo cabeçalho `X-Admin-Token` assinado.

Critérios de Aceite:
[ ] `record_user` sanitizado: apenas o admin pode alterar `plan` e `credits_minutes` via API.
[ ] `POST /api/affiliates/status` retorna HTTP 401 se token admin não for fornecido.
[ ] Testado que requisições forjadas do frontend não conseguem elevar privilégios.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] [Alta] Proteção contra Account Takeover e vazamento de PII de afiliados em /api/affiliates

Labels sugeridas: security, severity:alta, priority:p2

Descrição do Problema:
Em `src/app/api/affiliates/route.ts`:
1. A ação `GET ?action=me&affiliate_code=XYZ` permite consultar dados cadastrais completos de qualquer parceiro pelo código público (exibindo CPF, telefone, Pix e lista de compradores com e-mails).
2. A ação `POST ?action=affiliate_set_password` aceita `affiliate_code` e redefine a senha do afiliado sem pedir senha antiga ou token por e-mail.

Evidência Técnica (Arquivo:Linha):
src/app/api/affiliates/route.ts:478-485
```ts
} else if (userId || affiliateCode || emailParam) {
  affiliate = affiliates.find(a => ... a.affiliate_code === affiliateCode);
}
```
src/app/api/affiliates/route.ts:796-820
```ts
if (action === "affiliate_set_password") {
  const { affiliate_code, password } = body;
  aff.password_hash = hash;
}
```

Impacto: Vazamento grave de dados sensíveis protegidos pela LGPD e sequestro total de contas de afiliados.

Sugestão de Correção:
1. Exigir sessão autenticada (`getAuthenticatedAffiliate`) para visualização de métricas e dados pessoais em `action=me`.
2. No `affiliate_set_password`, permitir definição de senha apenas uma vez (quando `password_hash` for nulo) ou exigir token temporário criptografado enviado ao e-mail do titular.

Critérios de Aceite:
[ ] Consulta pública de métricas de afiliados desativada em `action=me`.
[ ] Redefinição de senha exige token verificado enviado ao e-mail cadastrado.
[ ] CPFs e telefones mascarados em qualquer resposta que não pertença estritamente à sessão do titular.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

Título: [Segurança] [Alta] Bloqueio de escrita direta de saldo pelo cliente no Supabase e imposição de RLS

Labels sugeridas: security, severity:alta, priority:p2

Descrição do Problema:
O componente `DashboardPage` contém chamadas diretas como `supabase.from('profiles').upsert({ credits_minutes, plan })` disparadas pelo navegador. Se as políticas de RLS permitirem update do próprio perfil, o usuário pode alterar seus minutos e plano localmente pelo console.

Evidência Técnica (Arquivo:Linha):
src/app/dashboard/page.tsx:164-169
```ts
await supabase.from("profiles").upsert({
  id: user.id,
  credits_minutes: activeCredits,
  plan: activePlan,
  updated_at: new Date().toISOString(),
});
```

Impacto: Fraude de créditos e uso infinito de recursos de inteligência artificial sem pagamento.

Sugestão de Correção:
1. No PostgreSQL do Supabase, remover permissão de escrita do usuário nas colunas `credits_minutes` e `plan` usando colunas protegidas ou triggers.
2. Atualizar saldos exclusivamente através de Stored Procedures RPC com `SECURITY DEFINER` e validação do Stripe.

Critérios de Aceite:
[ ] RLS do Supabase configurado bloqueando `UPDATE(credits_minutes, plan)` para a role `authenticated`.
[ ] Tentativa de update manual via DevTools rejeitada com código 42501 (permissão negada).
[ ] Concessão de créditos restrita ao processamento validado de webhooks da Stripe.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

Título: [Segurança] [Alta] Expurgo de arquivos com dados pessoais (PII) do histórico Git e adição ao .gitignore

Labels sugeridas: security, severity:alta, priority:p2

Descrição do Problema:
Arquivos em `data/*.json` (`data/users.json` e `data/affiliates.json`) contêm dados reais de clientes (nomes, e-mails, planos) e de afiliados (CPFs, números de telefone, chaves Pix e e-mails) e estão rastreados no Git.

Evidência Técnica (Arquivo:Linha):
Verificação de arquivos no repositório:
```bash
$ git ls-files data
data/affiliates.json
data/users.json
```

Impacto: Violação da LGPD (vazamento de dados pessoais e bancários de parceiros e clientes cadastrados).

Sugestão de Correção:
1. Adicionar `data/*.json` e `data/` ao `.gitignore` imediatamente.
2. Remover os arquivos do índice Git com `git rm --cached data/*.json`.
3. Se o repositório for compartilhado externamente, expurgar os commits que continham esses arquivos com `git-filter-repo`.

Critérios de Aceite:
[ ] `data/*.json` adicionado no `.gitignore`.
[ ] Arquivos de dados reais removidos do rastreamento de versão.
[ ] Criados arquivos de exemplo sanitizados (ex: `data/users.example.json`).

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

Título: [Segurança] [Média] Sanitização de templates de e-mail contra HTML Injection e sanitização de Markdown

Labels sugeridas: security, severity:media, priority:p3

Descrição do Problema:

1. Em `src/lib/email.ts`, dados fornecidos pelo usuário (`name`, `title`, `description`) são concatenados em HTML cru enviado por e-mail via `nodemailer`, permitindo injeção de tags HTML em clientes de e-mail.
2. Em `src/app/blog/[slug]/page.tsx`, o HTML gerado pelo `marked` é renderizado via `dangerouslySetInnerHTML` sem passar por `DOMPurify`.

Evidência Técnica (Arquivo:Linha):

````
src/lib/email.ts:78
```ts
<h1 style="...">Olá, ${cleanName}! Seja muito bem-vindo ao Peguei o Texto.</h1>
```
src/app/blog/[slug]/page.tsx:313
```tsx
<div className="blog-prose" dangerouslySetInnerHTML={{ __html: post.htmlContent || "" }} />
```
````

Impacto: Ataques de HTML Injection e Stored XSS em clientes de e-mail ou no navegador através de conteúdos markdown.

Sugestão de Correção:
1. Implementar função auxiliar `escapeHtml(str)` para substituir caracteres `& < > " '` antes de injetar variáveis em templates de e-mail.
2. Instalar biblioteca de sanitização (ex: `sanitize-html`) antes de repassar o HTML gerado pelo Markdown ao React.

Critérios de Aceite:
[ ] Função `escapeHtml` criada e aplicada a todas as interpolações de variáveis de usuário em `src/lib/email.ts`.
[ ] Sanitização de Markdown implementada no leitor de posts do blog.
[ ] Testado envio de sugestão com tags `<script>` e `<img>` verificando que o e-mail renderiza como texto puro.

--- FIM ISSUE 8 ---
